

Knowledge Representation

Lecture 1:

Introduction to Reasoning about Actions

dr Anna Maria Radzikowska

Wydział Matematyki i Nauk Informacyjnych

Pokój 504, gmach MiNI

E-mail: A.Radzikowska@mini.pw.edu.pl

Overview

- Reasoning about Actions and Changes
- Foundations to Classical Logic
 - Automated Theorem Proving – Resolution principle
- Models and types of knowledge
 - Epistemic logic
 - Dynamic and temporal logic
 - BDI systems
- Default Logic
- Rough Sets and Learning from Examples.

Reasoning about Actions

A *dynamic system* (DS) might be viewed as

- a collection of objects, together with their properties, and
- a collection of actions which, while performed, change properties of objects (in consequence, the state of the world).

One of the main tasks in modelling DS is the following:

Define reasoning methods that allow for deriving conclusions about necessary/possible results of performing actions.

The above task is the main objective in *Reasoning about Actions* (**RAC**).

Applications of RAC

- AI systems that operate in real words
- Design of autonomous agents (involving robotics, MAS)
- Planning tasks
- Model-based diagnosis
- Automatic control of complex systems
- ...

Ontologies of actions

- **Effects of actions:**

- ◇ deterministic (only one outcome), non-deterministic (at least two different outcomes)
- ◇ certain, typical (preferred)
- ◇ direct, indirect
- ◇ occurring immediately after an action ends; delayed effects
- ◇ all (some) effects are known.

Ontologies of actions (cont.)

- **Course of actions:**

- ◇ 1–step actions, actions with durations
- ◇ sequential actions, concurrent actions
- ◇ hierarchical occurrences (actions/subactions), non–hierarchical occurrences.

Ontologies of actions (cont.)

- **Course of actions:**

- ◇ 1–step actions, actions with durations
- ◇ sequential actions, concurrent actions
- ◇ hierarchical occurrences (actions/subactions), non–hierarchical occurrences.

- **Preconditions of actions:**

- ◇ if do not hold, the effects of an action are unknown (the effects are empty).

Main problems in RAC

- *Frame problem (McCarthy & Hayes, 1969):*

The difficulty is that of indicating and inferring all those things that do not change when actions are performed.

Main problems in RAC

- *Frame problem (McCarthy & Hayes, 1969):*

The difficulty is that of indicating and inferring all those things that do not change when actions are performed.

- *Ramification problem (Finger, 1987):*

Concerns the problem of concisely representing indirect effects of actions (propagation of changes).

It is usually unreasonable to explicitly enumerate all of the consequences of actions.

Main problems in RAC

- ***Frame problem (McCarthy & Hayes, 1969):***

The difficulty is that of indicating and inferring all those things that do not change when actions are performed.

- ***Ramification problem (Finger, 1987):***

Concerns the problem of concisely representing indirect effects of actions (propagation of changes).

It is usually unreasonable to explicitly enumerate all of the consequences of actions.

- ***Qualification problem (McCarthy, 1977):***

The number of preconditions of actions is usually immense and it is unreasonable (if ever possible) to explicitly enumerate and check all of (even most unlikely) possibilities.

Frame problem

Consider the following scenario (***Yale Shooting Problem***)

There is a shooter Bill and a turkey Fred. Initially Fred is alive and Bill has an unloaded gun. Bill can perform two actions: loading the gun (LOAD) and shooting the gun (SHOOT). Loading the gun makes the gun loaded, while shooting the gun makes it unloaded and, in addition, Fred is not alive anymore, provided that the gun was loaded.

Frame problem

Consider the following scenario (*Yale Shooting Problem*)

There is a shooter Bill and a turkey Fred. Initially Fred is alive and Bill has an unloaded gun. Bill can perform two actions: loading the gun (LOAD) and shooting the gun (SHOOT). Loading the gun makes the gun loaded, while shooting the gun makes it unloaded and, in addition, Fred is not alive anymore, provided that the gun was loaded.

Natural conclusions:

- after loading the gun it is loaded and Fred is still alive
- after shooting the gun it is unloaded and Fred is not alive.

Frame problem

Consider the following scenario (*Yale Shooting Problem*)

There is a shooter Bill and a turkey Fred. Initially Fred is alive and Bill has an unloaded gun. Bill can perform two actions: loading the gun (LOAD) and shooting the gun (SHOOT). Loading the gun makes the gun loaded, while shooting the gun makes it unloaded and, in addition, Fred is not alive anymore, provided that the gun was loaded.

Natural conclusions:

- after loading the gun it is loaded and Fred is still alive
- after shooting the gun it is unloaded and Fred is not alive.

However:

In classical logic we cannot infer that after loading the gun Fred is still alive!

Ramification problem

Consider the following modification of YSP:

There is a shooter Bill and a turkey Fred. Whenever Fred is walking, it is alive. Assume that the gun is loaded. Shooting the gun makes it unloaded and, if it was loaded before shooting, Fred is no longer alive.

Natural conclusion:

“After shooting the gun Fred is no longer alive and, on addition, it does not walk!”

In other words, “*Fred is not walking*” is the indirect effect of the shooting action.

Qualification problem

Consider the scenario *(Potato in the tailpipe problem, McCarthy 1977)*:

We want to start a car. Natural preconditions to do that are:

- the key must be turned in the ignition,
- there must be gas in the tank
- the battery must be connected
- the wiring must be intact
- ...

Also, there need not be a potato in the tailpipe!

Qualification problem

Consider the scenario *(Potato in the tailpipe problem, McCarthy 1977)*:

We want to start a car. Natural preconditions to do that are:

- the key must be turned in the ignition,
- there must be gas in the tank
- the battery must be connected
- the wiring must be intact
- ...

Also, there need not be a potato in the tailpipe!

Therefore, it is hardly possible (or practical) to check all of unlikely qualifications each time we are interested in using the car.

Simple dynamic systems

Assumptions:

- Inertia law
- Complete information about effects of actions
- Determinism
- Only direct effects of action
- Only sequential actions
- All actions are executable in every state
- If the precondition of an action does not hold, its effect is empty.

Simple dynamic systems

Assumptions:

- Inertia law
- Complete information about effects of actions
- Determinism
- Only direct effects of action
- Only sequential actions
- All actions are executable in every state
- If the precondition of an action does not hold, its effect is empty.

In order to represent this class of DS we will use action languages of the class $\mathcal{A}$.

Action Language $\mathcal{A}$

- Signature: $\Upsilon = (\mathcal{F}, \mathcal{A}c)$, where
 - $\mathcal{F}$ is a set of *fluents*
 - $\mathcal{A}c$ is a set of *actions*
- *Literal* – either a fluent f or its negation $\neg f$.

Notation:

For $f \in \mathcal{F}$, we write $\overline{f}$ to denote the literal corresponding to f , i.e. either f or $\neg f$.

Statements in $\mathcal{A}$

There are two types of statements in $\mathcal{A}$:

- **Value statement:** $\boxed{\overline{f} \text{ after } A_1, \dots, A_n}$
where $f \in \mathcal{F}$ and $A_i \in \mathcal{A}_c, i = 1, \dots, n$.

Intuitively, this statement is read as:

$\overline{f}$ holds after performing the sequence $A_1, \dots, A_n$ of actions in the initial state.

If $n = 0$, then we use the abbreviation $\boxed{\text{initially } \overline{f}}$ intuitively read as:

In the initial state $\overline{f}$ holds.

Statements in $\mathcal{A}$ (cont.)

● **Effect statement:** $A \text{ causes } \bar{f} \text{ if } \bar{g}_1, \dots, \bar{g}_k$

where $A \in \mathcal{Ac}$ and $f, g_1, \dots, g_k \in \mathcal{F}$.

If $k = 0$, then this statement is abbreviated to $A \text{ causes } \bar{f}$

Intuitively, these statements are respectively read as:

If the action A is performed in any state satisfying $g_1, \dots, g_k$, then in the resulting state $\bar{f}$ holds.

The performance of the action A leads to the state, where $\bar{f}$ is satisfied.

Any non-empty set D of sentences is called an **action domain**.

Example 1: Yale Shooting Problem (YSP)

Consider the following scenario:

There is a shooter Bill and a turkey Fred. Initially Fred is alive and Bill has an unloaded gun. Bill can perform two actions: loading the gun (LOAD) and shooting the gun (SHOOT). Loading the gun makes the gun loaded, while shooting the gun makes it unloaded and, in addition, Fred is not alive anymore, provided that the gun was loaded.

Representation in $\mathcal{A}$:

***initially** $\neg$ loaded;*

***initially** alive;*

*LOAD **causes** loaded;*

*SHOOT **causes** $\neg$ loaded;*

*SHOOT **causes** $\neg$ alive **if** loaded.*

Example 2: Stanford Murder Mystery (SMM)

Consider the following scenario:

There is a shooter Bill and a turkey Fred. Shooting the gun makes it unloaded and, in addition, Fred becomes dead, if the gun was loaded at the beginning of shooting. Initially Fred is alive. After shooting the gun Fred is dead.

Representation in $\mathcal{A}$:

initially *alive*;
SHOOT **causes** $\neg$ *loaded*;
SHOOT **causes** $\neg$ *alive* **if** *loaded*;
 $\neg$ *alive* **after** SHOOT.

Example 3: Stolen Car Scenario

Consider the following scenario:

A car was left for three consecutive days. Then it was found stolen.

Representation in $\mathcal{A}$:

initially $\neg stolen$;
stolen after WAIT, WAIT, WAIT.

Semantics of $\mathcal{A}$

Several auxiliary notions:

- **State**: any mapping $\sigma : \mathcal{F} \rightarrow \{0, 1\}$

For any $f \in \mathcal{F}$, if $\sigma(f) = 1$ then we say that f *holds in* σ and write $\sigma \models f$
 Σ stands for the set of all states.

- **Transition function**: any mapping $\Psi : \mathcal{A}c \times \Sigma \rightarrow \Sigma$.

For any $\sigma \in \Sigma$ and for every $A \in \mathcal{A}c$, $\Psi(A, \sigma)$ is the state resulting from performing the action A in the state σ .

Semantics of $\mathcal{A}$

Several auxiliary notions:

- **State:** any mapping $\sigma : \mathcal{F} \rightarrow \{0, 1\}$

For any $f \in \mathcal{F}$, if $\sigma(f) = 1$ then we say that f *holds in σ* and write $\sigma \models f$
 Σ stands for the set of all states.

- **Transition function:** any mapping $\Psi : \mathcal{A}_c \times \Sigma \rightarrow \Sigma$.

For any $\sigma \in \Sigma$ and for every $A \in \mathcal{A}_c$, $\Psi(A, \sigma)$ is the state resulting from performing the action A in the state σ .

A transition function is generalised to the mapping $\Psi^* : \mathcal{A}_c \times \Sigma \rightarrow \Sigma$ as follows:

- $\Psi^*(\varepsilon, \sigma) = \sigma$
- $\Psi^*((A_1, \dots, A_n), \sigma) = \Psi(A_n, \Psi^*(\sigma, (A_1, \dots, A_{n-1})))$.

Semantics of $\mathcal{A}$ (cont.)

$\mathcal{L}$ – an action language of the class $\mathcal{A}$ over the signature $\Upsilon = (\mathcal{F}, \mathcal{Ac})$.

Definition 1.1 A *structure for $\mathcal{L}$* is a pair $S = (\Psi, \sigma_0)$ such that Ψ is a transition function and $\sigma_0 \in \Sigma$ is the initial state.

Semantics of $\mathcal{A}$ (cont.)

$\mathcal{L}$ – an action language of the class $\mathcal{A}$ over the signature $\Upsilon = (\mathcal{F}, \mathcal{Ac})$.

Definition 1.1 A **structure for $\mathcal{L}$** is a pair $S = (\Psi, \sigma_0)$ such that Ψ is a transition function and $\sigma_0 \in \Sigma$ is the initial state.

Definition 1.2 Let $S = (\Psi, \sigma_0)$ be a structure for $\mathcal{L}$. We say that a statement s is **true in S** , in symbols $S \models s$, iff

- if s is of the form

$\overline{f}$ **after** $A_1, \dots, A_n$,

then $\Psi((A_1, \dots, A_n), \sigma_0) \models \overline{f}$

- if s is of the form

A **causes** $\overline{f}$ **if** $\overline{g_1}, \dots, \overline{g_k}$,

then for every $\sigma \in \Sigma$ such that $\sigma \models \overline{g_i}$, $i = 1, \dots, k$, it holds $\Psi(A, \sigma) \models \overline{f}$.

Semantics of $\mathcal{A}$ (cont.)

Definition 1.3 Let D be an action domain in the language $\mathcal{L}$ of a signature $\Upsilon = (\mathcal{F}, \mathcal{A}_c)$. A structure $S = (\Psi, \sigma_0)$ is a **model of D** iff

(M.1) for every $s \in D$, $S \models s$

(M.2) for every $A \in \mathcal{A}_c$, for every $f, g_1, \dots, g_k \in \mathcal{F}$, and for every $\sigma \in \Sigma$, if one of the following conditions holds:

(i) D contains an effect statement

A **causes** $\overline{f}$ **if** $\overline{g_1}, \dots, \overline{g_k}$,
and $\sigma \not\models \overline{g_i}$ for some $i = 1, \dots, k$

(ii) D does not contain an effect statement

A **causes** $\overline{f}$ **if** $\overline{g_1}, \dots, \overline{g_k}$

then $\sigma \models f \Leftrightarrow \Psi(\sigma, A) \models f$.

D is called **consistent** iff it has a model; otherwise it is **inconsistent**.

Queries in $\mathcal{A}$

By a *query* of $\mathcal{A}$ we mean a value statement in $\mathcal{A}$.

Definition 1.4 Given a action domain D , a query Q of $\mathcal{A}$ is a *consequence of* D , in symbols $D \models Q$, iff it is true in D .

Example 1: YSP (cont.)

$\Sigma = \{\sigma_0, \sigma_1, \sigma_2, \sigma_3\}$, where

$$\sigma_0 = \{alive, \neg loaded\}$$

$$\sigma_1 = \{alive, loaded\}$$

$$\sigma_2 = \{\neg alive, \neg loaded\}$$

$$\sigma_3 = \{\neg alive, loaded\}.$$

$$\Psi(\sigma_0, \text{LOAD}) = \sigma_1 \quad \text{by (M.1)}$$

$$\Psi(\sigma_0, \text{SHOOT}) = \sigma_0 \quad \text{by (M.2)}$$

$$\Psi(\sigma_1, \text{LOAD}) = \sigma_1 \quad \text{by (M.1)}$$

$$\Psi(\sigma_1, \text{SHOOT}) = \sigma_2 \quad \text{by (M.1)}$$

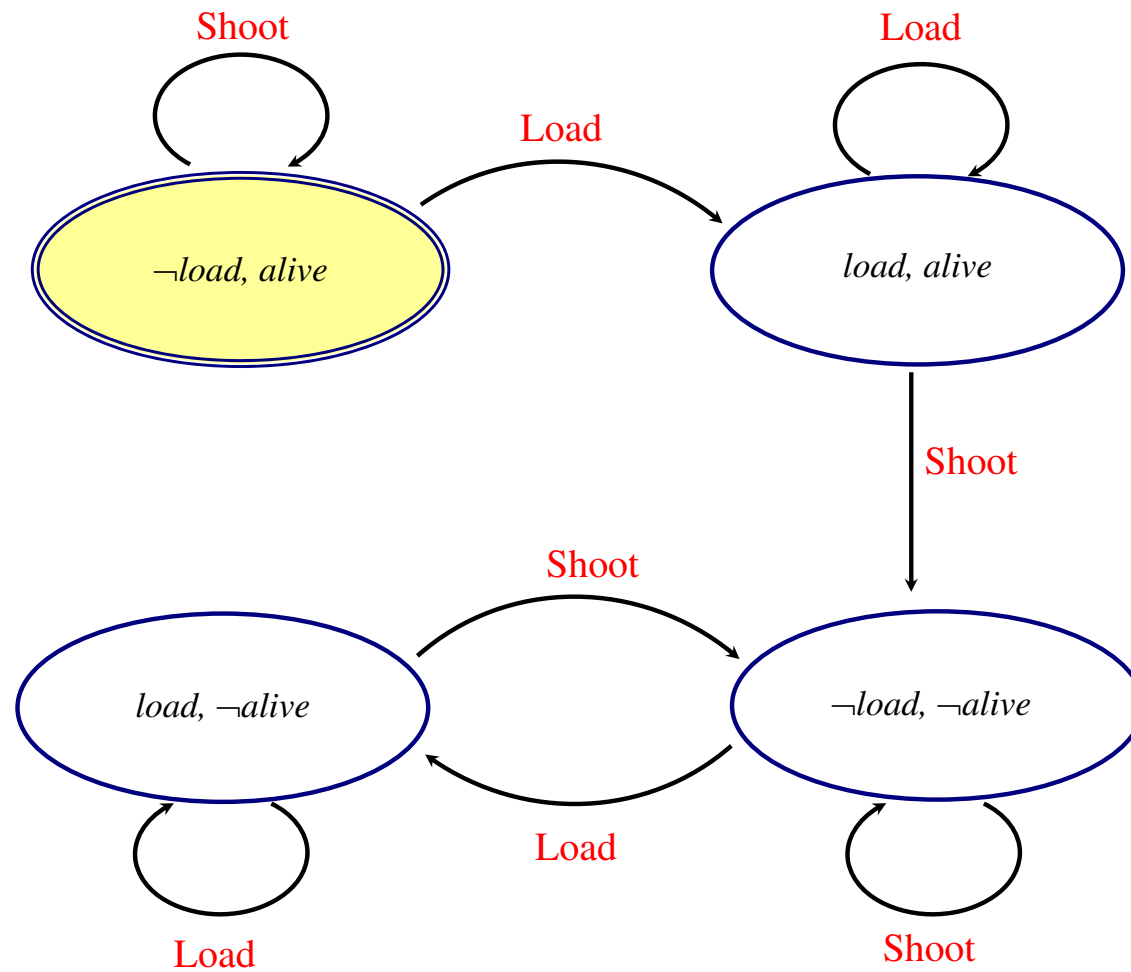
$$\Psi(\sigma_2, \text{LOAD}) = \sigma_3 \quad \text{by (M.1), (M.2)}$$

$$\Psi(\sigma_2, \text{SHOOT}) = \sigma_2 \quad \text{by (M.1), (M.2)}$$

$$\Psi(\sigma_3, \text{Load}) = \sigma_3 \quad \text{by (M.1), (M.2)}$$

$$\Psi(\sigma_3, \text{SHOOT}) = \sigma_2 \quad \text{by (M.1).}$$

Example 1: YSP (cont.)



Thank you for your attention!

Any questions are welcome.